

运算对象可按任意顺序求值

大多数运算符都没有规定运算对象的求值顺序（参见 4.1.3 节，第 123 页），这在一般情况下不会有什么影响。然而，如果一条子表达式改变了某个运算对象的值，另一条子表达式又要使用该值的话，运算对象的求值顺序就很关键了。因为递增运算符和递减运算符会改变运算对象的值，所以要提防在复合表达式中错用这两个运算符。

为了说明这一问题，我们将重写 3.4.1 节（第 97 页）的程序，该程序使用 for 循环将输入的第一个单词改成大写形式：

```
for (auto it = s.begin(); it != s.end() && !isspace(*it); ++it)
    *it = toupper(*it);    // 将当前字符改成大写形式
```

在上述程序中，我们把解引用 `it` 和递增 `it` 两项任务分开来完成。如果用一个看似等价的 while 循环进行代替

```
// 该循环的行为是未定义的！
while (beg != s.end() && !isspace(*beg))
    *beg = toupper(*beg++); // 错误：该赋值语句未定义
```

将产生未定义的行为。问题在于：赋值运算符左右两端的运算对象都用到了 `beg`，并且右侧的运算对象还改变了 `beg` 的值，所以该赋值语句是未定义的。编译器可能按照下面的任意一种思路处理该表达式：

```
*beg = toupper(*beg);    // 如果先求左侧的值
*(beg + 1) = toupper(*beg); // 如果先求右侧的值
```

也可能采取别的什么方式处理它。

4.5 节练习

150

练习 4.17：说明前置递增运算符和后置递增运算符的区别。

练习 4.18：如果第 132 页那个输出 `vector` 对象元素的 while 循环使用前置递增运算符，将得到什么结果？

练习 4.19：假设 `ptr` 的类型是指向 `int` 的指针、`vec` 的类型是 `vector<int>`、`ival` 的类型是 `int`，说明下面的表达式是何含义？如果有表达式不正确，为什么？应该如何修改？

```
(a) ptr != 0 && *ptr++          (b) ival++ && ival
(c) vec[ival++] <= vec[ival]
```

4.6 成员访问运算符

点运算符（参见 1.5.2 节，第 21 页）和箭头运算符（参见 3.4.1 节，第 98 页）都可用于访问成员，其中，点运算符获取类对象的一个成员；箭头运算符与点运算符有关，表达式 `ptr->mem` 等价于 `(*ptr).mem`：

```
string s1 = "a string", *p = &s1;
auto n = s1.size();    // 运行 string 对象 s1 的 size 成员
n = (*p).size();       // 运行 p 所指对象的 size 成员
n = p->size();          // 等价于 (*p).size()
```

因为解引用运算符的优先级低于点运算符，所以执行解引用运算的子表达式两端必须加上括号。如果没加括号，代码的含义就大不相同了：

```
// 运行 p 的 size 成员，然后解引用 size 的结果
*p.size();           // 错误：p 是一个指针，它没有名为 size 的成员
```

这条表达式试图访问对象 *p* 的 *size* 成员，但是 *p* 本身是一个指针且不包含任何成员，所以上述语句无法通过编译。

箭头运算符作用于一个指针类型的运算对象，结果是一个左值。点运算符分成两种情况：如果成员所属的对象是左值，那么结果是左值；反之，如果成员所属的对象是右值，那么结果是右值。

4.6 节练习

练习 4.20: 假设 *iter* 的类型是 `vector<string>::iterator`，说明下面的表达式是否合法。如果合法，表达式的含义是什么？如果不合法，错在何处？

- | | | |
|------------------------------------|-----------------------------|--------------------------------------|
| (a) <code>*iter++;</code> | (b) <code>(*iter)++;</code> | (c) <code>*iter.empty()</code> |
| (d) <code>iter->empty();</code> | (e) <code>++*iter;</code> | (f) <code>iter++->empty();</code> |

151

4.7 条件运算符

条件运算符 (`?:`) 允许我们把简单的 `if-else` 逻辑嵌入到单个表达式当中，条件运算符按照如下形式使用：

```
cond ? expr1 : expr2;
```

其中 *cond* 是判断条件的表达式，而 *expr1* 和 *expr2* 是两个类型相同或可能转换为某个公共类型的表达式。条件运算符的执行过程是：首先求 *cond* 的值，如果条件为真对 *expr1* 求值并返回该值，否则对 *expr2* 求值并返回该值。举个例子，我们可以使用条件运算符判断成绩是否合格：

```
string finalgrade = (grade < 60) ? "fail" : "pass";
```

条件部分判断成绩是否小于 60。如果小于，表达式的结果是 "fail"，否则结果是 "pass"。有点类似于逻辑与运算符和逻辑或运算符 (`&&` 和 `||`)，条件运算符只对 *expr1* 和 *expr2* 中的一个求值。

当条件运算符的两个表达式都是左值或者能转换成同一种左值类型时，运算的结果是左值；否则运算的结果是右值。

嵌套条件运算符

允许在条件运算符的内部嵌套另外一个条件运算符。也就是说，条件表达式可以作为另外一个条件运算符的 *cond* 或 *expr*。举个例子，使用一对嵌套的条件运算符可以将成绩分成三档：优秀 (high pass)、合格 (pass) 和不合格 (fail)：

```
finalgrade = (grade > 90) ? "high pass"
              : (grade < 60) ? "fail" : "pass";
```

第一个条件检查成绩是否在 90 分以上，如果是，执行符号?后面的表达式，得到 "high pass"；如果否，执行符号:后面的分支。这个分支本身又是一个条件表达式，它检查成绩是否在 60 分以下，如果是，得到 "fail"；否则得到 "pass"。